

Individual Reflective Report: Autonomous SAR Drone Project

Dmytro Chuprynyuk

AENGM0074 Group Design Project — University of Bristol — March 2026

1 Introduction

I was the software and computer vision lead for an autonomous SAR drone (AENGM0074 Group Design Project), responsible for the full software stack: state machine, vision system, search planning, target localisation, ground station, and testing infrastructure (~15,000 lines of Python). This report examines my contributions through AHEP4 learning outcomes [1]: design and problem-solving (M5), societal and environmental impact (M7), and teamwork and communication (M16, M17).

2 Design and Problem-Solving (M5)

2.1 Original Solutions to Complex Problems

The core challenge was building a fully autonomous mission on a Raspberry Pi 5 [2] with no GPU, operating at under 5 FPS, where software failures have physical consequences.

Dual-backend vision architecture. The vision system exposes a single interface—`detect_in_image(frame)`—while auto-detecting the platform and selecting the appropriate backend (Ultralytics on laptop, TFLite [3] on Pi). This abstraction let me add lens undistortion and model swapping without changing any other module.

Simulation-first development. Following Schön’s reflection-in-action principle [4], I built an interactive simulator with configurable GPS drift, camera shake, and inference throttling, proving every algorithm under pessimistic conditions before deployment.

Target localisation under uncertainty. Converting pixel detections to GPS coordinates required fusing camera geometry with drone telemetry via inverse-variance weighting ($w = 1/h^2$), with a bonus for near-centre detections where projection error is minimised. Video analysis revealed GPS timing lag (100–200 ms), causing ~1 m directional error at 5 m/s, though statistical averaging proved sufficient.

2.2 Safety-Critical Design

Working on a system that controls a physical aircraft shaped every design decision [5]. I designed a five-step progressive testing sequence—from Mission Planner waypoints (no custom code) through to full autonomy—following aerospace verification practices [6]. Each step adds exactly one variable, making failure causes unambiguous. Additional measures included automatic geofencing, RC kill switch priority, and headless-first field operation.

2.3 STEEPLE Considerations

Safety was addressed through progressive testing and geofencing; environmental impact through SSSI no-fly zone protection and choosing the 5 W Pi over GPU alternatives. The £60 platform makes SAR accessible to resource-limited organisations, legal compliance followed CAA regulations, and the ethical dimension of autonomous decision-making was handled by keeping a human in the verification loop.

3 Societal and Environmental Impact (M7)

SAR drones save lives by surveying large areas faster than ground teams and detecting casualties in inaccessible terrain [7, 8]. Even modest reductions in search time improve survival outcomes, particularly where a low-cost autonomous drone could replace expensive helicopter searches. However, the same technology enables unauthorised surveillance. I addressed this through human-in-the-loop verification—the system detects and localises, but a human operator decides what to do—and local-only image logging with no persistent surveillance capability. Given the dual-use potential of autonomous search and edge AI, responsible development requires transparency and design choices that prevent fully autonomous targeting.

Environmentally, automatic geofencing protects the SSSI’s rare bird habitats by switching to manual mode near the no-fly zone rather than relying on pilot awareness. The Pi 5 consumes ~5 W versus 15–30 W for GPU alternatives, and YOLOv8n was chosen as the smallest viable model. Simulation replaced approximately 50 physical flights, saving battery cycles and component wear. The drone platform is reusable for future cohorts, and the software is released under MIT licence.

4 Teamwork, Leadership, and Communication (M16, M17)

Our five-person team had distinct roles: I led software and CV, others handled hardware assembly, piloting, project administration, and flight dynamics research. In Belbin’s framework [9], I functioned as Specialist and Completer-Finisher—technical lead and de facto systems integrator. The team worked effectively where responsibilities had explicit handoff points: the hardware lead assembled the drone independently, consulting me only for Pi-to-Cube wiring; the pilot practised RC and understood abort procedures without engaging with code; the project manager booked field slots and submitted risk assessments I would have neglected. I adapted to constraints by front-loading integration when the hardware lead had exam commitments and prioritising software risk items to unblock the risk assessment submission.

In practice, the software was written by one person. This was not planned—it evolved because I had the strongest programming background and found it faster to build than onboard. Reflecting through Kolb’s cycle [10], I had optimised for output over team development. I attempted to lower the barrier with numbered test scripts, a simplified API, and documentation, but the combined context of MAVLink, state machines, and TFLite post-processing was too high for short-term engagement. The lesson: contribution points must be designed in from the start, not retrofitted.

The main technical disagreement—ROS 2 versus raw MAVLink—was resolved by a 20-line working heartbeat demo. In Tuckman’s model [11], this storming phase was brief because the team deferred to demonstrated results. Communicating across a significant skill gap was the hardest interpersonal challenge. The **simulator** proved the best communication device: teammates understood the system intuitively when they saw the drone flying, detecting, and descending on screen. The **ground station** translated complex state into large buttons usable outdoors, and **live demonstrations** at PDR and FDR generated more engagement than slides alone. Frustration over uneven workload was the most persistent tension, managed by focusing on the shared goal and explicitly acknowledging the project manager’s logistical contributions.

The most valuable feedback exchange: the pilot identified that abort and confirm-target buttons were too close—a genuine safety risk. I immediately separated the controls and added a confirmation dialog. Following Gibbs [12], this demonstrated that domain-specific expertise identifies risks invisible to the developer, and that designing for external feedback is itself an engineering discipline.

5 Self-Assessment and Development

5.1 Skills Developed

This project pushed me into unfamiliar domains. I learned MAVLink and ArduPilot [13, 14] from scratch—heartbeat negotiation, command acknowledgement, and autopilot configuration. Edge AI deployment (model export, TFLite [3] post-processing, Pi benchmarking) taught me that infrastructure around a model is where real complexity lies. Threading camera capture, inference, telemetry, and web streaming on a single-core Python process gave practical concurrency experience under GIL constraints. Field debugging on a Pi with no internet and limited screen space was perhaps the most underrated skill developed.

5.2 Time Management

Work was structured around weekly milestones: weeks 12–15 on simulation and the state machine, weeks 16–18 on Pi deployment, weeks 19–20 on field testing and retraining. A living ground-truth document, updated after every session, turned each “What’s Not Done Yet” list into the next session’s tasks. When weather cancelled our first flight, pre-planned bench tests kept the day productive.

5.3 Strengths and Weaknesses

My core strength is **rapid prototyping and system integration**—decomposing problems into bounded modules and integrating them quickly. The simulation-first methodology and living documentation scale directly to professional practice. I communicate well through demonstrations and visual tools, but less effectively in spontaneous verbal explanation.

My primary weakness is **designing for collaboration**: I build systems optimised for one person’s mental model rather than distributing work through simpler interfaces. I also **choose speed over inclusion**—building alone in two hours rather than onboarding a teammate in four. Effective for output, but it undermines team development. A third weakness is **delayed hardware testing**: weeks of laptop development before Pi deployment led to Python 3.13 incompatibilities and camera colour issues that cost days to resolve [6].

5.4 Development Programme

Based on reflective analysis using Gibbs’ cycle [12] and Kolb’s experiential learning model [10], I identify three development actions:

1. **Collaborative architecture**: define contribution points and interface contracts *before* implementation. Success metric: teammates contributing meaningful code without direct guidance.
2. **Day-one hardware deployment**: run a trivial script on target hardware within week one to surface platform issues early.
3. **Concise communication**: adopt a “one sentence, one diagram, one demo” framework for architecture explanations.

5.5 Connection to Career Goals

I came to Bristol from Ukraine to develop robotics skills I can bring back to help my country. This project directly involves technologies with humanitarian and defence applications—autonomous search, AI detection [15], and edge computing for field-deployable systems. The skills developed—MAVLink

integration, edge AI, safety-critical design [5]—transfer to autonomous systems Ukraine needs for mine clearance, reconnaissance, and disaster response.

If I had to summarise this project’s lesson: *the quality of an autonomous system is determined not by its algorithms’ cleverness, but by the rigour of its testing and the honesty of its failure analysis.*

References

- [1] Engineering Council. *The Accreditation of Higher Education Programmes (AHEP4)*. Tech. rep. <https://www.engc.org.uk/ahep>. Engineering Council UK, 2020.
- [2] Raspberry Pi Foundation. *Raspberry Pi 5*. <https://www.raspberrypi.com/products/raspberry-pi-5/>. Accessed: 2026. 2023.
- [3] Google. *TensorFlow Lite: Deploy Machine Learning Models on Mobile and Edge Devices*. <https://www.tensorflow.org/lite>. Accessed: 2026. 2023.
- [4] D. A. Schön. *The Reflective Practitioner: How Professionals Think in Action*. Basic Books, 1983.
- [5] N. G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- [6] P. Koopman. *How Safe Is Safe Enough? Measuring and Predicting Autonomous Vehicle Safety*. Technical report. Carnegie Mellon University, 2019.
- [7] M. Silvagni et al. “Multipurpose UAV for Search and Rescue Operations in Mountain Avalanche Events”. In: *Geomatics, Natural Hazards and Risk* 8.1 (2017), pp. 18–33.
- [8] M. A. Goodrich et al. “Supporting Wilderness Search and Rescue Using a Camera-Equipped Mini UAV”. In: *Journal of Field Robotics*. Vol. 25. 2008, pp. 89–110.
- [9] R. M. Belbin. *Team Roles at Work*. 2nd. Butterworth-Heinemann, 2010.
- [10] D. A. Kolb. *Experiential Learning: Experience as the Source of Learning and Development*. Prentice-Hall, 1984.
- [11] B. W. Tuckman. “Developmental Sequence in Small Groups”. In: *Psychological Bulletin* 63.6 (1965), pp. 384–399.
- [12] G. Gibbs. *Learning by Doing: A Guide to Teaching and Learning Methods*. Further Education Unit, Oxford Polytechnic, 1988.
- [13] ArduPilot Development Team. *ArduPilot: Open Source Autopilot*. <https://ardupilot.org/>. Accessed: 2026. 2024.
- [14] L. Meier, D. Honegger, and M. Pollefeys. *MAVLink: Micro Air Vehicle Communication Protocol*. <https://mavlink.io/>. Accessed: 2026. 2013.
- [15] G. Jocher, A. Chaurasia, and J. Qiu. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. Accessed: 2026. 2023.